dsp.LMSFilter System object

Package: dsp

LMS adaptive filter

Description

The `LMSFilter` implements an adaptive FIR filter object that returns the filtered output, the error vector, and filter weights. The LMS filter uses one of five different LMS algorithms.

To implement the adaptive FIR filter object:

1. Define and set up your adaptive FIR filter object. See Construction.
2. Call `step` to implement the filter according to the properties of `dsp.LMSFilter`. The behavior of `step` is specific to each object in the toolbox.

Note: Starting in R2016b, instead of using the `step` method to perform the operation defined by the System object™, you can call the object with arguments, as if it were a function. For example, `y = step(obj,x)` and `y = obj(x)` perform equivalent operations.

Construction

`lms = dsp.LMSFilter` returns an adaptive FIR filter object, `lms`, that computes the filtered output, filter error and the filter weights for a given input and desired signal using the Least Mean Squares (LMS) algorithm.

`lms = dsp.LMSFilter('PropertyName', PropertyValue,...)` returns an LMS filter object, `lms`, with each property set to the specified value.

`lms = dsp.LMSFilter(LEN,'PropertyName',PropertyValue,...)` returns an LMS filter object, `lms`, with the `Length` property set to `LEN`, and other specified properties set to the specified values.

Properties

`Method`	Method to calculate filter weights Specify the method used to calculate filter weights as `LMS`, `Normalized LMS`, `Sign-Error LMS`, `Sign-Data LMS`, or `Sign-Sign LMS`. The default is `LMS`.
`Length`	Length of FIR filter weights vector Specify the length of the FIR filter weights vector as a positive integer. The default is `32`.
`StepSizeSource`	How to specify adaptation step size Choose how to specify the adaptation step size factor as `Property` or `Input port`. The default is `Property`.
`StepSize`	Adaptation step size Specify the adaptation step size factor as a nonnegative real number. For convergence of the normalized LMS method, set the step size greater than `0` and less than `2`. This property only applies when the `StepSizeSource` property is `Property`. The default is `0.1`. This property is tunable.
`LeakageFactor`	Leakage factor used in LMS filter Specify the leakage factor as a real number between `0` and `1` inclusive. A leakage factor of `1` corresponds to no leakage in the adapting method. The default is `1`. This property is tunable.

`InitialConditions`	Initial conditions of filter weights Specify the initial values of the FIR filter weights as a scalar or vector of length equal to the `Length` property value. The default is `0`.
`AdaptInputPort`	Enable weight adaptation Specify when the LMS filter should adapt the filter weights. By default, the value of this property is `false`, and the object continuously updates the filter weights. When this property is set to `true`, an adaptation control input is provided to the `step` method. If the value of this input is nonzero, the object continuously updates the filter weights. If the input is zero, the filter weights remain at their current value.
`WeightsResetInputPort`	Enable weight reset Specify when the LMS filter should reset the filter weights. By default, the value of this property is `false`, and the object does not reset the weights. When this property is set to `true`, a reset control input is provided to the `step` method, and the `WeightsResetCondition` property applies. The object resets the filter weights based on the values of the `WeightsResetCondition` property and the `reset` input to the `step` method.
`WeightsResetCondition`	Reset trigger setting for filter weights Specify the event to reset the filter weights as `Rising edge`, `Falling edge`, `Either edge`, or `Non-zero`. The LMS filter resets the filter weights based on the values of this property and the `reset` input to the `step` method. This property only applies when the `WeightsResetInputPort` property is `true`. The default is `Non-zero`.
`WeightsOutput`	Enable returning filter weights Specify how to output the adapted filter weights as one of the following: • `'Last'` (default) — The object returns a column vector of weights corresponding to the last sample of the data frame. The length of the weights vector is the value given by the `Length` property. • `'All'` — The object returns a *FrameLength*-by-*Length* matrix of weights. The matrix corresponds to the full sample-by-sample history of weights values for all *FrameLength* samples of the input values. Each row in the matrix corresponds to a set of LMS filter weights calculated for the corresponding input sample. • `'None'` — This setting disables the weights output.

▸ Fixed-Point Properties

Methods

maxstep	Maximum step size for LMS adaptive filter convergence
msepred	Predicted mean-square error for LMS filter
msesim	Mean-squared error for LMS filter
reset	Reset filter states for LMS filter
step	Apply LMS adaptive filter to input

Common to All System Objects	
`clone`	Create System object with same property values
`getNumInputs`	Expected number of inputs to a System object
`getNumOutputs`	Expected number of outputs of a System object
`isLocked`	Check locked states of a System object (logical)

`release`	Allow System object property value changes

Examples

expand all

> **System Identification Using LMS Filter**

> **Cancel Noise Using LMS Filter**

> **Access the Full History of LMS Filter Weights**

Algorithms

This filter's algorithm is defined by the following equations.

$$y(n) = \mathbf{w}^T(n-1)\mathbf{u}(n)$$
$$e(n) = d(n) - y(n)$$
$$\mathbf{w}(n) = \alpha\mathbf{w}(n-1) + f(\mathbf{u}(n), e(n), \mu)$$

The various LMS adaptive filter algorithms available in this System object are defined as:

- LMS:

  $$f(\mathbf{u}(n), e(n), \mu) = \mu e(n)\mathbf{u}^*(n)$$

- Normalized LMS:

  $$f(\mathbf{u}(n), e(n), \mu) = \mu e(n)\frac{\mathbf{u}^*(n)}{\varepsilon + \mathbf{u}^H(n)\mathbf{u}(n)}$$

- Sign-Error LMS:

  $$f(\mathbf{u}(n), e(n), \mu) = \mu sign(e(n))\mathbf{u} * (n)$$

- Sign-Data LMS:

  $$f(\mathbf{u}(n), e(n), \mu) = \mu e(n) sign(\mathbf{u}(n))$$

  where **u**(n) is real.

- Sign-Sign LMS:

  $$f(\mathbf{u}(n), e(n), \mu) = \mu sign(e(n)) sign(\mathbf{u}(n))$$

  where **u**(n) is real.

The variables are as follows:

Variable	Description
n	The current time index
$\mathbf{u}(n)$	The vector of buffered input samples at step n
$\mathbf{u}^*(n)$	The complex conjugate of the vector of buffered input samples at step n
$\mathbf{w}(n)$	The vector of filter weight estimates at step n
$y(n)$	The filtered output at step n
$e(n)$	The estimation error at step n
$d(n)$	The desired response at step n

μ	The adaptation step size
α	The leakage factor (0 < α ≤ 1)

Extended Capabilities

C/C++ Code Generation

Generate C and C++ code using MATLAB® Coder™.

See Also

System Objects

`dsp.BlockLMSFilter` | `dsp.FIRFilter`

Blocks

LMS Filter

Introduced in R2012a